XENIX® System V/286 Operating System

File Formats(F)

Document Number 290190011-500-R00-0785

Contents

File Formats **(F)**

Name

intro – Introduction to file formats.

Description

This section outlines the formats of various files. Usually, these structures can be found in the directories **/usr/include** or **/usr/include/sys**.

The fields of the header structure are as follows:

c_type The type of the header.

c_date The date the backup was taken.

c_ddate The date the file system was backed up.

c_volume The current volume number of the backup.

c_tapea The current block number of this record. This is counting 512 byte blocks.

c_inumber The number of the inode being backed up if this is a TS_INODE type.

c_magic This contains the value MAGIC above, truncated as needed.

c_checksum This contains whatever value is needed to make the blocksum to CHECKSUM.

c_dinode This is a copy of the inode as it appears on the file system.

c_count This is the count of characters that follow describing the file. A character is zero if the block associated with that character was not present on the file system, otherwise the character is nonzero. If the block was not present on the file system no block was backed up and it is replaced as a hole in the file. If there is not sufficient space in this block to describe all of the blocks in a file, TS_ADDR blocks will be scattered through the file, each one picking up where the last left off.

c_addr This is the array of characters that is used as described above.

Each volume except the last ends with a tapemark (read as an end-of-file). The last volume ends with a TS_END block and then the tapemark.

The structure *idates* describes an entry of the file where backup history is kept.

See Also

backup(C), restore(C), filesystem(F)

Name

backup – Incremental dump tape format.

Description

The *backup*(C) and *restore*(C) commands are used to write and read incremental dump magnetic tapes.

The backup tape consists of a header record, some bit mask records, a group of records describing file system directories, a group of records describing file system files, and some records describing a second bit mask.

The header record and the first record of each description have the format described by the structure included by:

```
#include <dumprestor.h>
```

Fields in the *dumprestor* structure are described below.

NTREC is the number of 512 byte blocks in a physical tape record. MLEN is the number of bits in a bit map word. MSIZ is the number of bit map words.

The TS_ entries are used in the *c_type* field to indicate the type of header. The types and their meanings are as follows:

TS_TYPE	Tape volume label.
TS_INODE	A file or directory follows. The *c_dinode* field is a copy of the disk inode and contains bits telling the type of file.
TS_BITS	A bit mask follows. This bit mask has one bit for each inode that was backed up.
TS_ADDR	A subblock to a file (*TS_INODE*). See the description of *c_count* below.
TS_END	End-of-tape record.
TS_CLRI	A bit mask follows. This bit mask contains one bit for all inodes that were empty on the file system when backed up.
MAGIC	All header blocks have this number in *c_magic*.
CHECKSUM	Header blocks checksum to this value.

AR (F)

Name

ar – Archive file format.

Description

The archive command *ar* is used to combine several files into one. Archives are used mainly as libraries to be searched by the link editor *ld*(CP).

A file produced by *ar* has a *magic number* at the start, followed by the constituent files, each preceded by a file header. The magic number is 0177545 octal (or 0xff65 hexadecimal). The header of each file is declared in **/usr/include/ar.h**.

Each file begins on a word boundary; a NULL byte is inserted between files if necessary. Nevertheless the size given reflects the actual size of the file exclusive of padding.

Notice there is no provision for empty areas in an archive file.

See Also

ar(CP), ld(CP)

Name

acct – Format of per-process accounting file.

Description

Files produced as a result of calling *acct*(S) have records in the form defined by **<sys/acct.h>.**

In *ac_flag*, the AFORK flag is turned on by each *fork*(S) and turned off by an *exec*(S). The *ac_comm* field is inherited from the parent process and is reset by any *exec*. Each time the system charges the process with a clock tick, it also adds the current process size to *ac_mem* computed as follows:

(data size) + (text size) / (number of in-core processes using text)

The value of *ac_mem/ac_stime* can be viewed as an approximation to the mean process size, as modified by text-sharing.

See Also

acct(C), acct(S), acctcom(C)

Notes

The *ac_mem* value for a short-lived command gives little information about the actual size of the command, because *ac_mem* may be incremented while a different command (e.g., the shell) is being executed by the process.

Name

a.out – Format of assembler and link editor output.

Description

A.out is the output file of the assembler *as* and the link editor *ld*. Both programs will make **a.out** executable if there were no errors in assembling or linking, and no unresolved external references.

The format of **a.out**, called the **x.out** or segmented **x.out** format, is defined by the files **/usr/include/a.out.h** and **/usr/include/sys/relsym.h**. The **a.out** file has the following general layout:

1. Header.
2. Extended header.
3. File segment table (for segmented formats).
4. Segments (Text, Data, Symbol, and Relocation).

In the segmented format, there may be several text and data segments, depending on the memory model of the program. Segments within the file begin on boundaries which are multiples of 512 bytes as defined by the file's page size.

See Also

as(CP), ld(CP), nm(CP), strip(CP).

Name

checklist – List of file systems processed by *fsck*.

Description

The **/etc/checklist** file contains a list of the file systems to be checked when *fsck*(C) is invoked without arguments. The list contains at most 15 *special file* names. Each *special file* name must be on a separate line and must correspond to a file system.

See Also

fsck(C)

Name

core – Format of core image file.

Description

XENIX writes out a core image of a terminated process when any of various errors occur. See *signal*(S) for the list of reasons; the most common are memory violations, illegal instructions, bus errors, and user-generated quit signals. The core image is called **core** and is written in the process' working directory (provided it can be; normal access controls apply). A process with an effective user ID different from the real user ID will not produce a core image.

The first section of the core image is a copy of the system's per-user data for the process, including the registers as they were at the time of the fault. The size of this section depends on the parameter *usize*, which is defined in **/usr/include/sys/param.h**. The remainder represents the actual contents of the user's core area when the core image was written. If the text segment is read-only and shared, or separated from data space, it is not dumped.

The format of the information in the first section is described by the user structure of the system, defined in **/usr/include/sys/user.h**. The locations of registers are outlined in **/usr/include/sys/reg.h**.

See Also

adb(CP), setuid(S), signal(S)

Name

cpio – Format of cpio archive.

Description

The *header* structure, when the **c** option is not used, is:

```
struct {
        short   h_magic,
                h_dev,
                h_ino,
                h_mode,
                h_uid,
                h_gid,
                h_nlink,
                h_rdev,
                h_mtime[2],
                h_namesize,
                h_filesize[2];
        char    h_name[h_namesize rounded to word];
} Hdr;
```

When the **c** option is used, the *header* information is described by the statement below:

```
sscanf(Chdr,"%6o%6o%6o%6o%6o%6o%6o%11lo%6o%6o%s",
        &Hdr.h_magic,&Hdr.h_dev,&Hdr.h_ino,&Hdr.h_mode,
        &Hdr.h_uid,&Hdr.h_gid,&Hdr.h_nlink,&Hdr.h_rdev,
        &Longtime,&Hdr.h_namesize,&Longfile,Hdr.h_name);
```

Longtime and *Longfile* are equivalent to *Hdr.h_mtime* and *Hdr.h_filesize*, respectively. The contents of each file is recorded in an element of the array of varying length structures, *archive*, together with other items describing the file. Every occurrence of *h_magic* contains the constant 070707 (octal). The items *h_dev* through *h_mtime* have meanings explained in *stat*(S). The length of the null-terminated pathname *h_name*, including the NULL byte, is given by *h_namesize*.

The last record of the *archive* always contains the name TRAILER!!!. Special files, directories, and the trailer are recorded with *h_filesize* equal to zero.

See Also

cpio(C), find(C), stat(S)

Name

dir – Format of a directory.

Syntax

#include <sys/dir.h>

Description

A directory behaves exactly like an ordinary file, except that no user may write into a directory. The fact that a file is a directory is indicated by a bit in the flag word of its inode entry (see *filesystem*(F)). The structure of a directory is given in the include file **/usr/include/sys/dir.h**.

By convention, the first two entries in each directory are "dot" (**.**) and "dotdot" (**..**). The first is an entry for the directory itself. The second is for the parent directory. The meaning of "dotdot" is modified for the root directory of the master file system; there is no parent, so "dotdot" has the same meaning as "dot".

See Also

filesystem(F)

Name

filesystem – Format of a system volume.

Syntax

#include <sys/filsys.h>
#include <sys/types.h>
#include <sys/param.h>

Description

Every file system storage volume (e.g., a hard disk) has a common format for certain vital information. Every such volume is divided into a certain number of blocks. The number of bytes in a block is implementation dependent as defined by BSIZE in **/usr/include/sys/param.h**. Block 0 is unused and is available to contain a bootstrap program or other information.

Block 1 is the *super-block*. The format of a super-block is described in **/usr/include/sys/filesys.h**. In that include file, *S_isize* is the address of the first data block after the i-list. The i-list starts just after the super-block in block 2; thus the i-list is *s_isize*–2 blocks long. *S_fsize* is the first block not potentially available for allocation to a file. These numbers are used by the system to check for bad block numbers. If an ''impossible'' block number is allocated from the free list or is freed, a diagnostic is written on the console. Moreover, the free array is cleared so as to prevent further allocation from a presumably corrupted free list.

The free list for each volume is maintained as follows. The *s_free* array contains, in *s_free*[1], . . ., *s_free*[*s_nfree*–1], up to 49 numbers of free blocks. *S_free*[0] is the block number of the head of a chain of blocks constituting the free list. The first long in each free-chain block is the number (up to 50) of free-block numbers listed in the next 50 longs of this chain member. The first of these 50 blocks is the link to the next member of the chain. To allocate a block: decrement *s_nfree*, and the new block is *s_free*[*s_nfree*]. If the new block number is 0, there are no blocks left, so give an error. If *s_nfree* becomes 0, read in the block named by the new block number, replace *s_nfree* by its first word, and copy the block numbers in the next 50 longs into the *s_free* array. To free a block, check if *s_nfree* is 50; if so, copy *s_nfree* and the *s_free* array into it, write it out, and set *s_nfree* to 0. In any event set *s_free*[*s_nfree*] to the freed block's number and increment *s_nfree*.

S_tfree is the total free blocks available in the file system.

S_ninode is the number of free i-numbers in the *s_inode* array. To allocate an inode: if *s_ninode* is greater than 0, decrement it and return *s_inode*[*s_ninode*]. If it was 0, read the i-list and place the numbers of all free inodes (up to 100) into the *s_inode* array, then try again. To free an inode, provided *s_ninode* is less than 100, place its number into *s_inode*[*s_ninode*] and increment *s_ninode*. If *s_ninode* is already 100, do not bother to enter the freed inode into any table. This list of inodes only speeds up the allocation process. The information about whether the inode is really free is maintained in the inode itself.

S_tinode is the total number of free inodes available in the file system.

S_flock and *s_ilock* are flags maintained in the core copy of the file system while it is mounted and their values on disk are immaterial. The value of *s_fmod* on disk is also immaterial, and is used as a flag to indicate that the super-block has changed and should be copied to the disk during the next periodic update of file system information.

S_ronly is a read-only flag used to indicate write-protection.

S_time is the last time the super-block of the file system was changed, and is a double-precision representation of the number of seconds that have elapsed since 00:00 Jan. 1, 1970 (GMT). During a reboot, the *s_time* of the super-block for the root file system is used to set the system's idea of the time.

I-numbers begin at 1, and the storage for inodes begins in block 2. Also, inodes are 64 bytes long, so 8 of them fit into a block. Therefore, inode *i* is located in block (*i*+15)/8, and begins 64×((*i*+15) (mod 8)) bytes from its start. Inode 1 is reserved for future use. Inode 2 is reserved for the root directory of the file system, but no other i-number has a built-in meaning. Each inode represents one file. For the format of an inode and its flags, see *inode*(F).

Files

/usr/include/sys/filsys.h

/usr/include/sys/stat.h

See Also

fsck(C), inode(F), mkfs(C)

Name

gettydefs – Speed and terminal settings used by getty.

Description

The **/etc/gettydefs** file contains information used by *getty*(M) to set up the speed and terminal settings for a line. It supplies information on what the *login* prompt should look like. It also supplies the speed to try next if the user indicates the current speed is not correct by typing a character.

Each entry in **/etc/gettydefs** has the following format:

label#initial-flags#final-flags# login-prompt#next-label

Each entry is followed by a blank line. The various fields can contain quoted characters of the form **\b**, **\n**, **\c**, etc., as well as *\nnn*, where *nnn* is the octal value of the desired character. The various fields are:

label Identifies the **/etc/gettydefs** entry to *getty*. This could be a letter or number. The label corresponds to the line mode field in **/etc/ttys**. *Init* passes the line mode as an argument to *getty*.

initial-flags Sets the initial *ioctl*(S) settings if a terminal type is not specified to *getty*. The flags that *getty* understands are the same as the ones listed in *tty*(M). Normally only the speed flag is required in the *initial-flags*. *Getty* automatically sets the terminal to raw input mode and takes care of most of the other flags. The *initial-flag* settings remain in effect until *getty* executes *login*(M).

final-flags Sets the same values as the *initial-flags*. These flags are set just prior to *getty* executes *login*. The speed flag is again required. The composite flag **SANE** takes care of most of the other flags that need to be set so that the processor and terminal are communicating in a rational fashion. The other two commonly specified *final-flags* are **TAB3**, so that tabs are sent to the terminal as spaces, and **HUPCL**, so that the line is hung up on the final close.

login-prompt Contains login prompt message that greets users. Unlike the above fields where white space is ignored (a SPACEBAR, TAB, or NEWLINE), it is included in the *login-prompt* field.

next-label Identifies the next entry in **gettydefs** for *getty* to try if the current one is not successful. *Getty* tries the next label if a user presses the BREAK key while attempting to log in to the system. Groups of entries, for example, for dial-up lines or for lines, should form a closed set so that *getty* cycles back to the original entry if none of the entries is successful. For instance, **1200** is linked to **300**, which in turn is linked to **1200**, which finally is linked to **2400**.

If *getty* is called without a second argument, then the first entry of **/etc/gettydefs** is used, thus making the first entry of **/etc/gettydefs** the default entry. The first entry is also used if *getty* can not find the specified *label*. If **/etc/gettydefs** itself is missing, there is one entry built into the command which will bring up a terminal at **300** baud.

After you modify **/etc/gettydefs**, run it through *getty* with the check option to be sure there are no errors.

Files

/etc/gettydefs

See Also

getty(M), ioctl(S), login(M)

Name

helplist – Format of the Help database

Description

Help uses an ASCII database file each time you execute *help* without any arguments. Each line in the database represents a line on a menu. Indented lines represent lines that are on the submenu to the previous line. Each submenu line is indented two more spaces than the previous line. All lines beginning in the same column are on the same menu, until *help* encounters a line that is indented less than the previous line.

The innermost indented lines are manual page description. These lines alsc include the section that contains the command (C, CP, CT, S) and a brief description of the command. For example:

```
This is a top level menu #1 (starting in column 0)
  This is a submenu under the top menu #1 (column 2)
    cmd C A sample command with the manual section (column 4)
    cmd C Another sample command on the same menu
  Another submenu under the top menu #1 (column 2)
    morecmds CP more commands
Another top level menu
```

If the manual page command is followed by an asterisk (*) and another command, *help* looks at the manual page for the command that follows the asterisk. This is useful when there are two commands documented on the same page. *Help* does not display the asterisk, the second command, or the section. For example, the system library function **toupper** is on the same manual page as **conv**. The line for **toupper** is:

toupper*conv S Converts characters to upper case

Help displays:

toupper Converts characters to upper case

Notes

You cannot mix commands and submenus on the same level.

Files

/usr/bin/help – the help command

/usr/lib/Help/helplist – the default help command database

See Also

help(C)

Name

inode – Format of an inode.

Syntax

#include <sys/types.h>
#include <sys/ino.h>

Description

An inode for a plain file or directory in a file system has the structure defined by **<sys/ino.h>**. For the meaning of the defined types *off_t* and *time_t* see *types*(F).

Files

/usr/include/sys/ino.h

See Also

filesystem(F), stat(S), types(F)

The first line (@s) contains the number of lines inserted/deleted/unchanged respectively. The second line (@d) contains the type of the delta (currently, normal: D, and removed: R), the SCCS ID of the delta, the date and time of creation of the delta, the login name corresponding to the real user ID at the time the delta was created, and the serial numbers of the delta and its predecessor, respectively.

The @i, @x, and @g lines contain the serial numbers of deltas included, excluded, and ignored, respectively. These lines are optional.

The @m lines (optional) each contain one MR number associated with the delta; the @c lines contain comments associated with the delta.

The @e line ends the delta table entry.

User Names

The list of login names and/or numerical group IDs of users who may add deltas to the file, separated by NEWLINES. The lines containing these login names and/or numerical group IDs are surrounded by the bracketing lines @u and @U. An empty list allows anyone to make a delta.

Flags

Keywords used internally (see *admin*(CP) for more information on their use). Each flag line takes the form:

@f <*flag*> <*optional text*>

The following flags are defined:

@f t <*type of program*>
@f v <*program name*>
@f i
@f b
@f m <*module name*>
@f f <*floor*>
@f c <*ceiling*>
@f d <*default-sid*>
@f n
@f j
@f l <*lock-releases*>
@f q <*user defined*>

The **t** flag defines the replacement for the %Y% identification keyword. The **v** flag controls prompting for MR numbers in addition to comments; if the optional text is present it defines an MR number validity checking program. The **i** flag controls the warning/error aspect of the "No id keywords" message. When the **i** flag is not present, this message is only a warning; when the **i** flag is present, this message will cause a "fatal" error (the file will not be gotten, or the delta will not be made). When the **b** flag is present the **–b** option may be used with the *get* command to

Name

sccsfile – Format of an SCCS file.

Description

An SCCS file is an ASCII file. It consists of six logical parts: the *checksum*, the *delta table* (contains information about each delta), *user names* (contains login names and/or numerical group IDs of users who may add deltas), *flags* (contains definitions of internal keywords), *comments* (contains arbitrary descriptive information about the file), and the *body* (contains the actual text lines intermixed with control lines). Each logical part of an SCCS file is described in detail below.

Throughout an SCCS file there are lines which begin with the ASCII SOH (start of heading) character (octal 001). This character is hereafter referred to as the *control character* and will be represented graphically as @. Any line described below which is not depicted as beginning with the control character should not begin with the control character. Entries of the form DDDDD represent a five digit string (a number between 00000 and 99999).

Checksum

The checksum is the first line of an SCCS file. The form of the line is:

@hDDDDD

The value of the checksum is the sum of all characters, except those on the first line. The @hR provides a *magic number* of (octal) 064001.

Delta Table

The delta table consists of a variable number of entries of the form:

@s DDDDD/DDDDD/DDDDD
@d <type> <**SCCS ID**> yr/mo/da hr:mi:se <pgmr> DDDDD DDDDD
@i DDDDD ...
@x DDDDD ...
@g DDDDD ...
@m <MR number>
.
.
@c <comments> ...
.
.
@e

MNTTAB (F)

Name

mnttab – Format of mounted file system table.

Syntax

#include <stdio.h>
#include <mnttab.h>

Description

The **/etc/mnttab** file contains a table of devices mounted by the *mount*(C) command.

Each table entry contains the pathname of the directory on which the device is mounted, the name of the device special file, the read/write permissions of the special file, and the date on which the device was mounted.

The maximum number of entries in **mnttab** is based on the system parameter NMOUNT located in **/usr/sys/conf/c.c**, which defines the number of allowable mounted special files.

See Also

mount(C)

If a parameter has no default value, an explicit specification for the parameter must be given in the description file. See *config*(CP) for a list of the tunable parameters.

See Also

config(CP)

Fields 11-14: maximum of four interrupt vector addresses. Each address is followed by a unique letter or a space.

Devices that are not interrupt-driven have an interrupt vector size of zero. Devices which generate interrupts but are not of the standard character or block device mold, should be specified with a type (field 4) which has neither the block nor character bits set.

Part 2

This part contains definitions for the system line discipline. Each line has nine fields. Each field is a maximum of eight characters delimited by a space if less than eight:

Field 1:	Device associated with this line
Field 2:	Open routine
Field 3:	Close routine
Field 4:	Read routine
Field 5:	Write routine
Field 6:	Ioctl routine
Field 7:	Receiver interrupt routine
Field 8:	Transmitter interrupt routine
Field 9:	Modem control interrupt routine

Part 3

This part contains definitions for device aliases. Each line has 2 fields:

Field 1:	Alias name of device (8 chars. maximum)
Field 2:	Reference name of device as given in part 1 (8 chars. maximum)

Aliases may be used in place of actual device names when creating the *config*(CP) description file.

Part 4

This part contains the names and default values for tunable parameters. Each line has 2 or 3 fields:

Field 1:	Parameter name to be used in the *config*(CP) description file (20 chars. maximum)
Field 2:	Parameter name as it will appear in the resulting **c.c** file (20 chars. maximum)
Field 3:	Default parameter value (20 chars. maximum)

Name

master – Master device information table.

Description

This file is used by the *config*(CP) program to obtain device information that enables it to generate the configuration files. The file consists of 4 parts, each separated by a line with a dollar sign ($) in column 1. Part 1 contains device information; part 2 contains the line discipline table; part 3 contains names of devices that have aliases; and part 4 contains tunable parameter information. Any line with an asterisk (*) in column 1 is treated as a comment.

Part 1

This part contains definitions for the system devices. Each line has 14 fields with the fields delimited by tabs and/or spaces:

Field 1: device name (8 chars. maximum).
Field 2: interrupt vector size (decimal, in bytes).
Field 3: device mask (octal). Each "on" bit indicates that the driver has the corresponding handler or structure:
- 000400 not used
- 000200 not used
- 000100 initialization handler
- 000040 not used
- 000020 open handler
- 000010 close handler
- 000004 read handler
- 000002 write handler
- 000001 ioctl handler.

Field 4: device type indicator (octal):
- 000200 not used
- 000100 init handler
- 000040 not used
- 000020 required device
- 000010 block device
- 000004 character device
- 000002 not used
- 000001 not used.

Field 5: handler prefix (4 chars. maximum).
Field 6: not used.
Field 7: major device number for block device.
Field 8: major device number for character device.
Field 9: maximum number of devices per controller (decimal).
Field 10: not used.

cause a branch in the delta tree. The **m** flag defines the first choice for the replacement text of the %M% identification keyword. The **f** flag defines the ''floor'' release; the release below which no deltas may be added. The **c** flag defines the ''ceiling'' release; the release above which no deltas may be added. The **d** flag defines the default SID to be used when none is specified on a *get* command. The **n** flag causes *delta* to insert a ''null'' delta (a delta that applies *no* changes) in those releases that are skipped when a delta is made in a *new* release (e.g., when delta 5.1 is made after delta 2.7, releases 3 and 4 are skipped). The absence of the **n** flag causes skipped releases to be completely empty. The **j** flag causes *get* to allow concurrent edits of the same base SID. The **l** flag defines a *list* of releases that are *locked* against editing (*get*(CP) with the **–e** option). The **q** flag defines the replacement for the %Q% identification keyword.

Comments

Arbitrary text is surrounded by the bracketing lines @t and @T. The comments section typically contains a description of the file's purpose.

Body

The body consists of text lines and control lines. Text lines don't begin with the control character, but control lines do. There are three kinds of control lines: *insert*, *delete*, and *end*, as follows:

```
@I DDDDD
@D DDDDD
@E DDDDD
```

The digit string (DDDDD) is the serial number corresponding to the delta for the control line.

See Also

admin(CP), delta(CP), get(CP), prs(CP)

XENIX *Programmer's Guide*

Name

stat – Data returned by stat system call.

Syntax

#include <sys/stat.h>

Description

The **sys/stat.h** include file contains the definition for the structure returned by the *stat* and *fstat* functions. The structure is defined as:

```
struct stat {
        dev_t   st_dev;     /* ID of device containing directory entry */
        ino_t   st_ino;     /* inode number */
        ushort  st_mode;    /* file mode */
        short   st_nlink;   /* # of links */
        ushort  st_uid;     /* owner uid */
        ushort  st_gid;     /* owner gid */
        dev_t   st_rdev;    /* device ID (block and char devices only) */
        off_t   st_size;    /* file size in bytes */
        time_t  st_atime;   /* time of last access */
        time_t  st_mtime;   /* time of last data modification */
        time_t  st_ctime;   /* time of last file status 'change' */
};
```

Note that the *st_atime*, *st_mtime*, and *st_ctime* values are measured in seconds since 00:00:00 (GMT) on January 1, 1970.

The *st_mode* value is actually a combination of one or more of the following file mode values:

```
S_IFMT          0170000     /* type of file */
    S_IFDIR     0040000     /* directory */
    S_IFCHR     0020000     /* character special */
    S_IFBLK     0060000     /* block special */
    S_IFREG     0100000     /* regular */
    S_IFIFO     0010000     /* fifo */
    S_IFNAM     0050000     /* name space entry */
S_INSEM         01          /* XENIX semaphore */
S_INSHD         02          /* XENIX shared memory */
S_ISUID         04000       /* set user ID on execution */
S_ISGID         02000       /* set group ID on execution */
S_ISVTX         01000       /* save swapped text even after use */
S_IREAD         00400       /* read permission, owner */
S_IWRITE        00200       /* write permission, owner */
S_IEXEC         00100       /* execute/search permission, owner */
```

Files

/usr/include/sys/stat.h

See Also

stat(S)

Name

types – Primitive system data types.

Syntax

#include <sys/types.h>

Description

The data types defined in the include file **<sys/types.h>** are used in XENIX system code; some data of these types are accessible to user code.

The form *daddr_t* is used for disk addresses except in an inode on disk. (See *filesystem*(F).) Times are encoded in seconds since 00:00:00 GMT, January 1, 1970. The major and minor parts of a device code specify kind and unit number of a device and are installation-dependent. Offsets are measured in bytes from the beginning of a file. The *label_t* variables are used to save the processor state while another process is running.

See Also

filesystem(F)

Name

utmp, wtmp – Formats of utmp and wtmp entries.

Syntax

#include <sys/types.h>
#include <utmp.h>

Description

These files, which hold user and accounting information for such commands as *who*(C), *write*(C), and *login*(M), have the following structure as defined by **<utmp.h>**:

```
#define   UTMP_FILE     "/etc/utmp"
#define   WTMP_FILE     "/etc/wtmp"
#define   ut_name       ut_user

struct utmp {
        char        ut_user[8];         /* User login name */
        char        ut_id[4];           /* Usually line # */
        char        ut_line[12];        /* Device name (console, lnxx) */
        short       ut_pid;             /* Process id */
        short       ut_type;            /* Type of entry */
        struct      exit_status {
          short       e_termination;    /* Process termination status */
          short       e_exit;           /* Process exit status */
        } ut_exit;                      /* The exit status of a process
                                         * Marked as DEAD_PROCESS. */
        time_t      ut_time;            /* Time entry was made */
};

/* Definitions for ut_type */
#define EMPTY           0
#define RUN_LVL         1
#define BOOT_TIME       2
#define OLD_TIME        3
#define NEW_TIME        4
#define INIT_PROCESS    5               /* Process spawned by "init" */
#define LOGIN_PROCESS   6               /* A "getty" process waiting for login */
#define USER_PROCESS    7               /* A user process */
#define DEAD_PROCESS    8
#define ACCOUNTING      9
#define UTMAXTYPE       ACCOUNTING  /* Largest legal value of ut_type */
```

```
/* Special strings or formats used in the "ut_line" field when */
/* Accounting for something other than a process */
/* No string for the ut_line field can be more than 11 chars + */
/* A NULL in length */
#define RUNLVL_MSG "run-level %c"
#define BOOT_MSG "system boot"
#define OTIME_MSG "old time"
#define NTIME_MSG "new time"
```

Files

/usr/include/utmp.h
/etc/utmp
/etc/wtmp

See Also

getut(S), login(C), who(C), write(C)

Index - File Formats(F)